WM9M4 Assignment Report

Part 1: Rasteriser Optimisation

Introduction:

Given a basic CPU rasteriser, the objective of this part of the project was to optimise it as much as possible, first using only one CPU core, then using multiple cores using parallelisation. There are two given scenes for testing, and a third one to construct.

The third scene was chosen to have as many triangles as possible with very round spheres, and some squares to make sure the renderer is still working properly. This should make per-triangle optimisations less effective (as most triangles are much smaller) but parallel and overall optimisations more effective.

All optimisation tests are performed in the same conditions on the same machine which is plugged in and with the window in focus. All tests run at least 10 loops, although the total number of loops is variable (generally lower for slower render cycles). The first loop of every test is always ignored (its rendering time was too often more than 5% different compared to every other loop). A loop for the first and third scene is defined as the camera moving forward and backwards fully once. A loop for the second scene is defined as the sphere moving right and left fully once. The unit of every optimisation test is seconds.

Optimisation:

As the compiler already does a lot of single threaded optimisation (and can even do some vectorisation and parallelisation given the right circumstances, although these were turned off for the tests), the objective is to find parts of the given code which were not being optimized (sufficiently) by the compiler. To do so most effectively, the code was run with CPU profiling enabled, which meant the assembly instructions the CPU was working on were sampled repeatedly and an approximation of how long each function took to compute in the overall program was created. This revealed the fact that the triangle “draw” function was taking around 70% of the CPU time, with almost half of that being spent on calculating barycentric coordinates with the “getCoordinates” function.

Optimisation 1: Pre-Calculating Barycentric Coordinates

A simplified version of the draw triangle function could be written in pseudo-code as:

```
draw() {
    getCameraBounds();
    if (triangle is very small) { return; }
    for (x in camera_bounds) {
        for (y in camera_bounds) {
            getCoordinates(x, y);  // i.e., barycentric coordinates
            if (barycentric coordinates indicate (x, y) is outside
                the triangle) { continue; }
            getColour(x, y);
            getNormal(x, y);
            getDepth(x, y);
            if (depth > buffer.depth(x, y)) { continue; }
            computeShader();
            drawToBuffer();
        }
    }
}.
```

Since the "getCoordinates" function was taking the most amount of time when profiling the program, it will be optimized first. Instead of calling it once for every pixel, we will move it outside of the loop and pre-compute the barycentric coordinates for every pixel. This requires calculating the x and y derivates for the three barycentric coordinates α, β, γ:

$$\frac{\partial\alpha(x,y)}{\partial x}, \frac{\partial\beta(x,y)}{\partial x}, \frac{\partial\gamma(x,y)}{\partial x}, \frac{\partial\alpha(x,y)}{\partial y}, \frac{\partial\beta(x,y)}{\partial y}, \frac{\partial\gamma(x,y)}{\partial y}.$$

Since all three barycentric coordinates (and therefore their derivatives) are calculated in the same way (with different vertices of the triangle), we focus here on the two partial derivates of α. Since $\alpha(x,y)$ is linear with respect to x and to y and it is separable (i.e., $\exists f(x), g(y)$ such that $\alpha(x,y) = f(x) + g(y)$), we know that its partial derivatives with respect to x and y are constant. We obtain

$$\frac{\partial\alpha(x,y)}{\partial x} = \pm(v_y - w_y),$$

$$\frac{\partial \alpha(x,y)}{\partial y} = \mp(v_x - w_x)$$

where v_x, v_y, w_x and w_y are the x and y coordinates of the two opposite vertices to the barycentric coordinate we are calculating, the sign is dependent on the order of vertices for the triangle, either clockwise or counterclockwise.

Table 1: Barycentric Pre-Calculation Optimization Run Times

	Initial Test	Bary Opt.	Speedup
Scene 1	4.665	3.042	53.37%
Scene 2	1.827	1.081	69.00%
Scene 3	11.007	10.142	8.53%

Now that the barycentric coordinates and their derivatives are pre-calculated, each loop of x can add the x derivatives to all three barycentric coordinates, and the same for each loop of y with the y derivatives. This gives significantly faster rendering (as seen in table 1), especially for scene 1 and 2. As suspected, scene 3 does not benefit as much from this change as most triangles are very small, therefore they would only have had a couple of loops where they called "getCoordinates", instead of hundreds of loops.

Handling Very Small Triangles on Scene 3:

Scene 3 also introduced a bug with the original rasterizer. The initial renderer was not built to handle very small triangles and purposefully ignores them in the draw function, as seen in the pseudo-code above, if they have an area smaller than 1. This causes serious rendering artefacts, especially when far away from those triangles (as seen in figure 1).

This is fixed by removing the "if" statement quitting out of the function early. Scenes 1 and 2 run sightly slower without this check by less than 5%; scene 3 runs slower by 15 to 20 percent but is now free of artefacts.

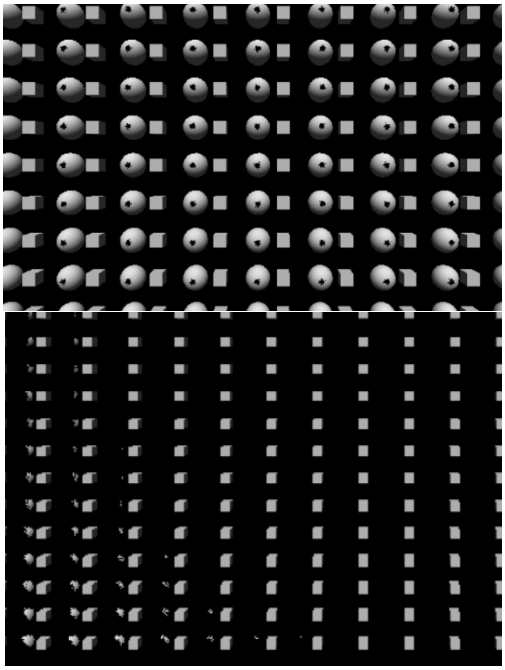

Figure 1: Two images of rendering artefacts caused by ignoring small triangles in scene 3

Therefore, an alternate scene 3 runtime will now also be calculated for future optimizations; while the scene is the same, the draw call will not be ignoring these small triangles and will display the correct scene. As comparison, the run times for all scenes without ignoring small triangles is present in table 2.

Table 2: The rendering time of all 3 scenes w/ & w/o the barycentric optimization and w/ & w/o ignoring small triangles

	Initial Test	w/o ignore	Slowdown
Scene 1	4.665	4.848	3.78%
Scene 2	1.827	1.871	2.33%
Scene 3	11.007	13.098	15.96%
	Opt. 1	w/o ignore	Slowdown
Scene 1	3.042	3.109	2.18%
Scene 2	1.081	1.114	2.95%
Scene 3	10.142	12.294	17.50%

Optimisation 2: Pre-Calculating Triangle Edges

Now that the barycentric coordinates are pre-calculated, the draw loop does not need to test every pixel inside the square bounds of the triangle. Instead of looping on both x, and y, we will only loop on y and pre-calculate the bounds on x for that specific y. This will allow us to skip the barycentric coordinate check as we will already know we are inside the triangle.

For a given pixel x in a y loop, we have the following condition on each barycentric coordinate (here only with α as the other two are equivalent with their respective derivatives)

$$\alpha \geq 0 \Leftrightarrow \alpha_0 + \frac{\partial \alpha}{\partial x}(x - x_0) \geq 0$$

where x_0 is the pixel at which α_0 was calculated (the start of the x loop, where y is fixed). We can isolate x to find the conditions on x for α to be greater or equal to 0. The equation can be rearranged as

$$\frac{\partial \alpha}{\partial x}(x - x_0) \geq -\alpha_0 \Rightarrow \begin{cases} \frac{\partial \alpha}{\partial x} > 0, x \geq -\alpha_0 \div \frac{\partial \alpha}{\partial x} + x_0 \\ \frac{\partial \alpha}{\partial x} < 0, x \leq -\alpha_0 \div \frac{\partial \alpha}{\partial x} + x_0 \\ \frac{\partial \alpha}{\partial x} = 0, \begin{cases} \alpha_0 \geq 0, true \\ \alpha_0 < 0, false \end{cases} \end{cases}$$

which gives a condition on x based on the sign of the partial derivative. If we restrict x based on all three barycentric coordinates, the loop will be entirely contained inside of the triangle, removing the need for the check.

As seen in table 3, compared to the previous optimisation, scenes 1 and 2 benefit further from this optimization to the "draw" function. However, scene 3 does not benefit from this change as the triangles are too small. Pre-calculating the bounds on x is more work than simply looping over the entire bounds for such small triangles.

Table 3: Pre-calculated x-bounds optimization run times

	Optim. 1	Optim. 2	Speedup
Scene 1	3.042	2.523	20.55%
Scene 2	1.081	0.873	23.81%
Scene 3	10.142	10.677	-5.00%
Alt Sc. 3	12.294	12.219	0.62%

Optimisation 3: Small Optimisations to "draw"

There are a few more small optimisations that can be done to speed-up the draw function by removing work from the loop. Light normalisation, present in the loop at first, can be computed as the function starts. The same can be said about a few constant calculations over the triangle; by removing them from the loop, we save the CPU a few computations per loop.

We also move the calculations from inside the depth test "if" statement to outside it; this is to avoid the negative effects of incorrect branch predictions and let the program continue instead of backtracking on those calculations as they do not modify any values for the next loop anyway. This specific branching optimisation did not change run time much at all, except perhaps a speedup of around 1%.

Table 4: Draw function small optimisation run times

	Optim. 2	Optim. 3	Speedup
Scene 1	2.523	2.145	17.65%
Scene 2	0.873	0.732	19.34%
Scene 3	10.677	10.419	2.47%
Alt Sc. 3	12.219	12.016	1.68%

The results of such small changes are surprising (as seen in table 4), with more than a 15% speedup compared to the previous optimisation. Scene 3, as usual, does not benefit as much from these changes.

Parallelisation:

To make the code concurrent, the first tested idea was to make a simple mesh allocator which gave the next mesh to whichever thread became available. This was done with an atomic integer counter which was incremented each time a thread fetched its value; that value told the thread which index of the vector of meshes the thread should now work on. No data racing protection was given to the depth buffer nor to the colour buffer, yet all three scenes rendered perfectly, and faster, with no visual artefacts, flickering, or incorrect rendering.

Changing the number of threads did not introduce any rendering errors, nor did adding even more overlapping objects to the scene. Running the program on an entirely different machine also did not introduce any rendering errors. This parallelisation did

however introduce noticeable frame stutters, but whether those are caused by this parallelisation implementation or simply a kernel scheduling issue is difficult to determine. However, even with those frame stutters, the run time was still better than the single-threaded code.

Other parallelisation implementations were attempted anyway. One such attempt made one renderer per thread, with its own depth buffer and pixel buffer. However, this was extremely slow, partly due to the memory allocation required per frame, but mostly because of the triple for loop required to write all buffers back to the main buffer with a depth check every loop. The two other attempts were simpler but resulted in even slower code: a mutex would be acquired when writing to the renderer's depth buffer and canvas. One attempt had a single mutex, while another had one mutex per pixel. This resulted in millions of locks being acquired per frame, which is far too costly compared to the single-threaded code.

With the original parallelisation attempt working correctly and faster, it was kept. The Visual Studio compiler also offers automatic vectorisation (use of SIMD instructions) and parallelisation of certain loops, for which the final run times for the project were calculated with (those were turned off for all other optimisations).

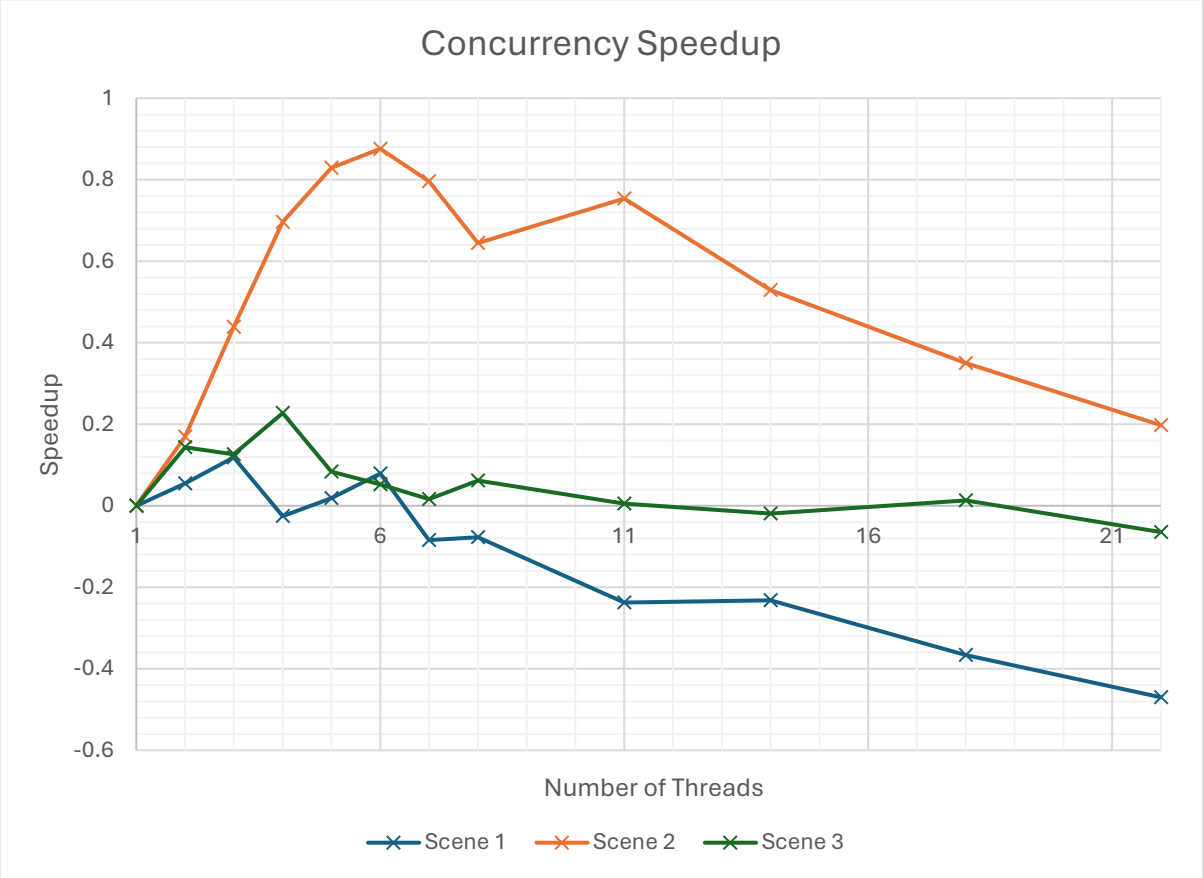

Figure 2: The speedup compared to single threaded code depending on the number of threads

As seen in figure 2, the speedup was greatest with fewer number of threads, with scene 1 best at 3 threads, scene 3 at 4, and scene 1 at 6. Scene 3 was potentially predicted to

have the best speedup, and while it does benefit slightly from concurrency, it is not that much faster than the single-threaded code. However, it does not seem to have as much of a slowdown compared to the other scenes when adding more threads (likely because of the large number of meshes in the scene compared to the other 2 scenes).

Conclusion:

The optimisations to the "draw" function along with the parallelisation have managed to give a total performance improvement of over 350% for scene 2 (as seen in table 5). Scene 1 had a significant speedup, albeit not as high as scene 2. Scene 3 was purposefully made to get the least amount of speedup, and indeed only benefitted by 25-30%. Some more improvements could be made, particularly to how triangles are created every frame as the current method is rather inefficient, and for parallelisation, but this would require significantly more code.

Table 5: Final Run Times and Speedup

	Initial Test	Final Test	Speedup
Scene 1	4.665	1.987	134.77%
Scene 2	1.827	0.390	368.49%
Scene 3	11.007	8.489	29.66%
Alt Sc. 3	13.098	10.261	27.64%

The parallelisation working without data protection on the depth buffer and pixel buffer is rather surprising, but the rendering was perfectly correct, even if the speedup was not equal to the number of threads (although this is of course never expected either way).

Part 2: Chat Application

Introduction:

The chat app is a basic messaging application containing a Graphical User Interface (GUI) on the client side which can be used to send messages to other clients, both in a global chat room, and individually to other clients. The client sends the message to the server, which then sends that message either to every client (including the client who sent it) or only to the 2 clients concerned if it is a direct message.

The application is written in C++ for Windows specifically and would require reimplementation to work on a different system (both because of the GUI using DirectX, and because of the network implementation using WinSock). It uses the Transmission Control Protocol (TCP) for communication between individual clients and the server. Clients never communicate with each other directly; all communication is done through the server.

Network Implementation:

Each client has two threads, one to receive messages from the server, and one to handle all the other tasks. This is because the function to receive messages from the

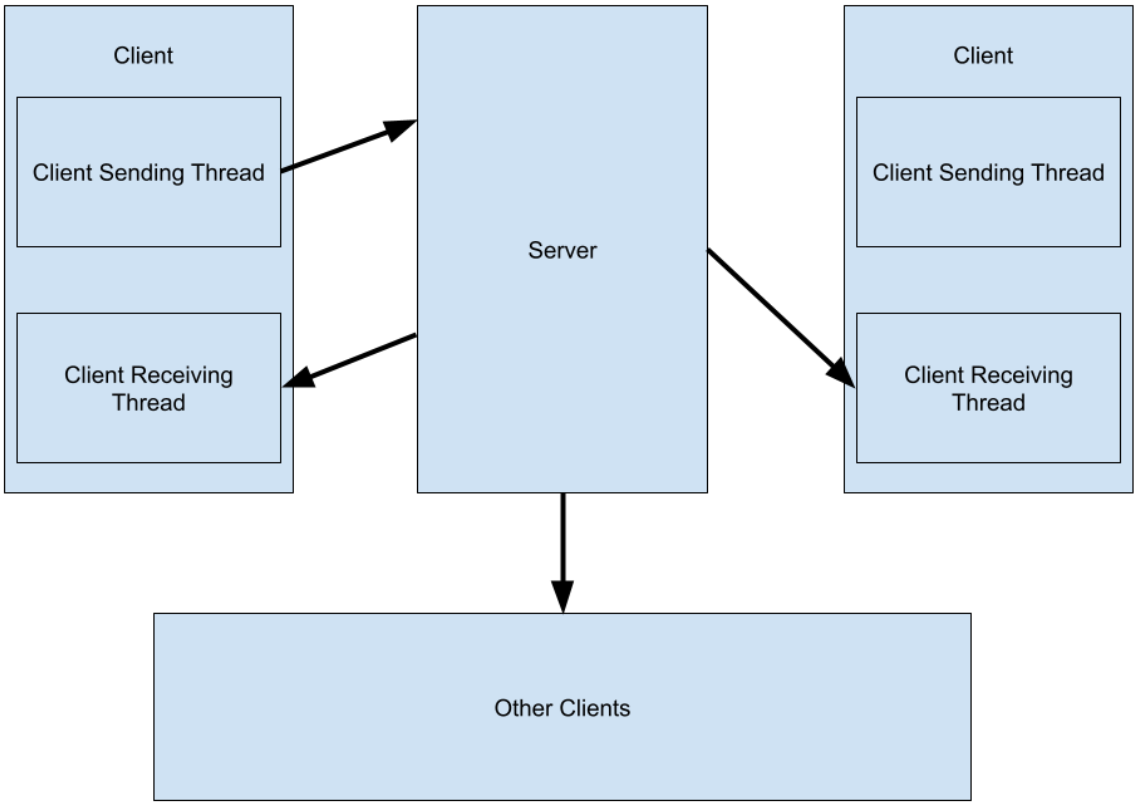

Diagram 1: How a single message sent by a client is sent over the network

server is blocking, which means no other tasks can be performed while waiting for a message. When the user decides to send a message from the GUI, that same thread calls the send function and sends the message to the server. When the receiving thread receives a message, it parses it (the format of outgoing and incoming messages is explained below) and then adds the corresponding message to the message queue to be read by the GUI. This message queue is lock-protected, which while not strictly required for the client, is a consequence of the symmetry of how the message queue is handled on both the client and server as they both use the same implementation.

The server has two threads per connected client, one to receive messages from that client and one to send messages to them. The threads are closed when the client disconnects or the TCP connection fails in some way. When receiving a message, the thread parses it and then adds it to the queue (as mentioned above, the queue uses a lock to ensure only one message is added at a time). Each sending thread is busy waiting for a message (which is a potential implementation improvement by adding a conditional variable) at which point it correctly formats the message and sends it to its corresponding client. Another potential improvement is having only one thread handle sending the messages to every client.

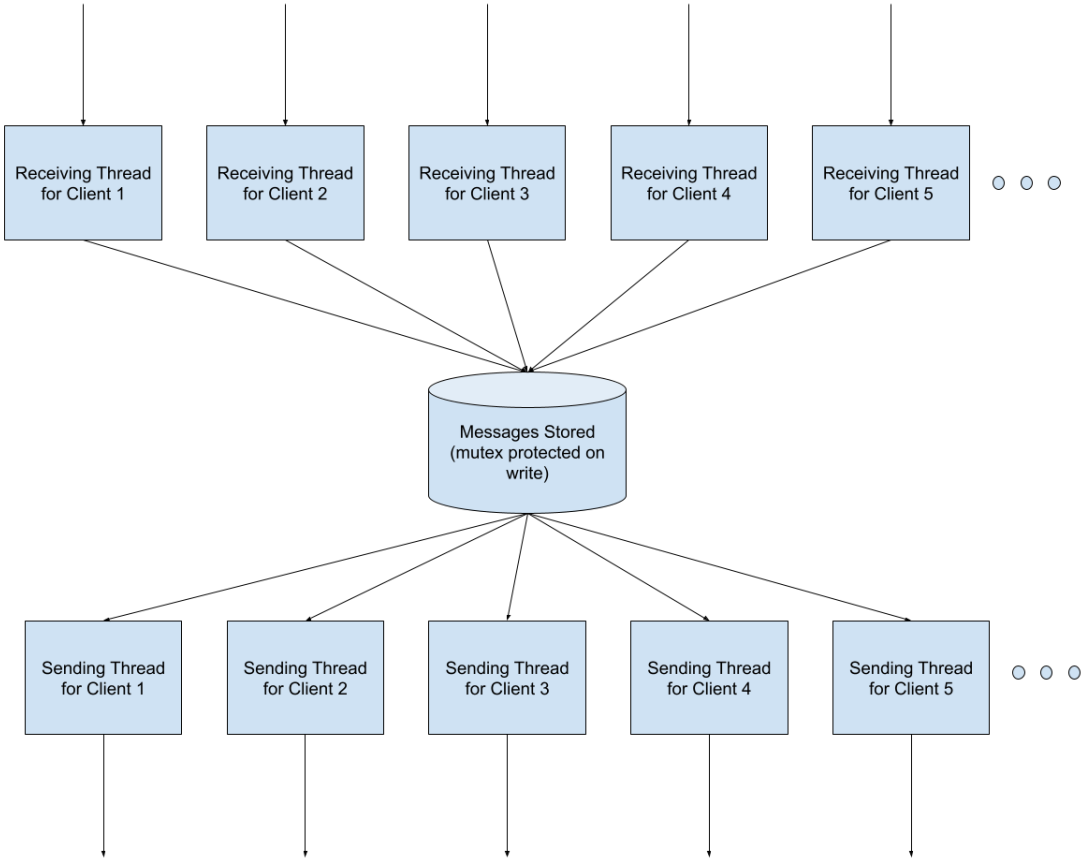

Diagram 2: How the server handles threads and the messaging queue

The message format is as follows, where square brackets [] represent optional parts of the message. When sending a message, the client sends

```
[:dm_username: ]message
```

to the server, where “dm_username” is the username of whom the message is intended for if it is a direct message (this part is omitted if it is not a direct message) and “message” is the user’s message. This also means that the user may send a direct message in the main chat room if they use the colon command with an appropriate username (the message is lost if the username does not exist). When sending a message to a client, the server sends

```
username (timestamp):[:dm_username:] message
```

to the client, where username is the sender’s username and timestamp is the message’s timestamp (currently only used to handle whether the message is valid or not but could be extended to display when the message was sent to the user). The part of the message after the first colon is identical to what the original client sent (except for an added space after the colon if it is not a direct message).

After being parsed, messages are stored (both on the Server and Client, due to the implementation being the same) as the following struct:

```
struct ChatMessage {
    char buffer[512] = "";
    char user[32] = "";
    int timestamp = -1;
    bool is_dm = false;
};
```

which can then be read by the individual threads on the server to send messages, or by the GUI on the client side to display messages appropriately.

Graphical User Interface:

The graphical user interface runs in DirectX12 with ImGui and was built upon the DirectX12 example file provided by ImGui. The main thread rebuilds the user interface every frame, storing data from previous frames to provide a seamless experience to the user. The main window is split into three parts: the chat room, the user input, and the user list.

The chat room displays messages sequentially from when they were received. Each frame, all the messages stored are read and added as a line of text. The user list, stored as a C++ set to avoid duplicates, is also constructed from these messages every frame. A connection and disconnection message is sent for each client appropriately, but since the message backlog is not sent when a client connects, the client may receive a message from another client not in their list, at which point the username is added to the set.

To send a message, the user can type in the text box underneath the chat room display. The buffer saves their input between frames and when the user presses enter, the send function is called with the contents of the buffer to send the message to the server. To know when the user has pressed the enter key, a specific ImGui flag is used which makes the function return true for the frame when the enter key has been pressed.

Finally, the user list is displayed on the right from all the collected connection and disconnection messages, as well as additional messages if some clients connected

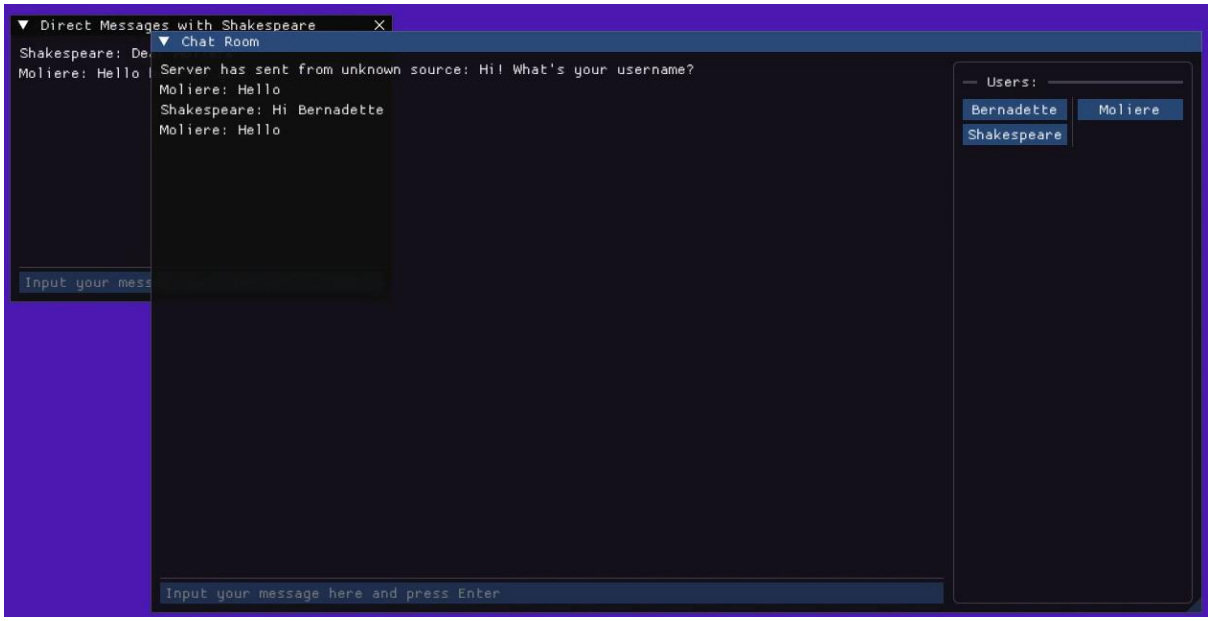

Figure 3: The ImGui GUI for the Chat Application

before the current client connected. Each username can be clicked on to open a separate window with a similar format to the main window, except without the user list. The separate window can be used to send direct messages to other clients, bypassing the main chat room. The user list section takes a fixed amount of space no matter the size of the overall Chat Room window. The chat message and send input sections take the rest of the space on the left.

When choosing which messages should be read in a specific direct message window, the sending user and receiving user for each message are checked against the corresponding username for the concerned window; if either of them match (and the username of the window is not the same as the username of the client), then we display the message in that window. If the direct message window is that of the client (i.e., they are sending messages to themselves), we check that both the recipient and the sending users are the same username as the client.

Conclusion:

The chat application works reasonably well, with both general and direct messages being implemented. For future work, the code would likely benefit from a slight rework on the server side to avoid opening as many threads. The application protocol and message storing would also benefit from slightly more dynamic code, including putting the length of the message when sending over the network. There are also security risks as messages are not encrypted, and setting the same username as someone else would also allow a client to receive another user's direct messages as well. But the point was to experiment with networking and graphical user interfaces and make a functioning chat application, and at that it was a decent success.